

CSC343HY Tutorial 7

Relational Algebra

Relational Algebra (RA) is a *procedural* query language: you specify *how* to compute a result by composing operators step by step. Every operator takes one or more relations as input and produces a relation as output, so operators can be nested freely. Today you will practice translating natural-language questions into RA expressions using the library schema below.

1. Operator Quick Reference

Symbol	Name	What it does
$\sigma_{\text{cond}}(R)$	Select	Keeps rows of R satisfying <i>cond</i> .
$\pi_{A_1, \dots, A_n}(R)$	Project	Keeps only the listed columns (removes duplicates).
$R \bowtie S$	Natural Join	Joins R and S on all shared column names.
$R \bowtie_{\text{cond}} S$	Theta Join	Joins on an explicit condition.
$\rho_{B/A}(R)$	Rename	Renames column A to B in R .
$\delta(R)$	Dedup	Removes duplicate tuples from R .
$\tau_A(R) / \tau_{-A}(R)$	Sort	Sorts by A ascending / descending.
$\gamma_{G; f(A)}(R)$	Group/Agg	Groups by G , computes aggregate f over A .
$R \cup S$	Union	All tuples in R or S (schemas must match).
$R \cap S$	Intersection	Tuples in both R and S .
$R - S$	Difference	Tuples in R but not S .
$R \times S$	Cartesian Product	All combinations of tuples from R and S .

Order of operations (highest to lowest precedence): (1) σ, π, ρ ; (2) $\times, \bowtie$; (3) $\cap$; (4) $\cup, -$. Use parentheses freely to override precedence and improve clarity.

2. The Schema

Today's exercises use a **Community Library** database with four relations. Primary keys are underlined; foreign keys are **bold**.

Relation	Attributes	Notes
book	<u>isbn</u> , title, author_id , copy_type	copy_type $\in$ {hardcopy, digital}
member	<u>utorid</u> , name, role	role $\in$ {Instructor, Student, Staff}
author	<u>author_id</u> , name, genera	
renting	<u>rent_id</u> , utorid , isbn , rent_date, return_date, overdue, fine	return_date is NULL until returned; overdue is boolean

Join hint. renting $\bowtie$ member joins on utorid; renting $\bowtie$ book joins on isbn; book $\bowtie$ author joins on author_id. When a natural join would match on the wrong columns, use a theta join with

an explicit condition.

3. Worked Example

Question: Find the utorids of all members who have ever rented a digital book.

Step 1: identify the data you need. The answer lives in **renting** (who rented what) and **book** (which books are digital).

Step 2: filter first. $\sigma_{copy_type='digital'}(book)$

Step 3: join to get the renters. $\sigma_{copy_type='digital'}(book) \bowtie renting$

Step 4: project to keep only what is asked.

$$\pi_{utorid}(\sigma_{copy_type='digital'}(book) \bowtie renting)$$

4. Exercises

Work through these with your neighbour. Write your RA expression in the space provided, then be ready to discuss with the class.

Warm-Up: Single-Table Queries

1. Find the utorids of the members who rented the book with ISBN 1234.

2. List the names of all authors whose genre (genera) is 'Fantasy'.

3. List all members' utorids ordered by their name in descending order.

Joins

4. List the names of all authors whose books have been rented at least once.

5. List the **names** of all students (**role** = 'Student') who have at least one overdue book. Ignore duplicates.

6. Find the **titles** of all books that have *never* been rented. *Hint: think about set difference.*

Aggregation

7. Find the total fine owed by the member with **utorid** = 'ABCD'. Return the result with the column renamed to **Total Fine**.

8. Find the number of instructors who currently have overdue books that have not yet been returned (**return_date** is NULL and **overdue** is **true**).

Trickier Problems

9. For each member, find the total number of books they have rented. Return `utorid` and the count, calling it `num_rentals`.

10. Find the `utorids` of members who have rented books by *more than one* distinct author. *Hint: you will need a join, a group-by with COUNT, and a selection on the aggregate.*

11. **Challenge.** Can you write an RA expression that finds the names of all members who are *also* authors in the `author` table? Discuss with your neighbour whether this is possible given the current schema, and explain your reasoning.

5. Common Pitfalls

1. Applying σ *after* π when the filter column was projected away. Filter first, then project.
2. Forgetting that natural join matches on *all* shared column names. If two tables share a column you do *not* want to join on, rename one copy first with ρ .
3. Using $\cup$ or $\cap$ on relations with different schemas, the schemas must be identical.
4. Applying $-$ (set difference) when you mean a σ with a NOT condition, difference operates on

two full relations, not a row-level filter.

5. Confusing γ (aggregate) with π (project). You cannot compute **SUM** or **COUNT** with π alone.
6. Trying to join tables that share no common column or whose only shared column has different semantics, check the schema before writing a natural join.

Before You Leave

Checklist

1. I can apply σ to filter rows and π to project columns.
2. I know the difference between a natural join and a theta join, and when to use each.
3. I can chain multiple operators together and read them inside-out.
4. I can use γ to group rows and compute aggregates.
5. I can use $\cup$, $\cap$, and $-$ on union-compatible relations.
6. I can explain why Question 11 may or may not be solvable with the given schema.